



Assignment No. 01

Computer Organization & Assembly Language (EE-2003)

Deadline: 18th September 2024

ID: _____

NAME: _____

SECTION: _____

Read the Instructions Carefully

- ❖ Assignments are designed for your learning copying will lead to plagiarism. Practicing the questions yourself will **help you in your exams**. Fill the **memory and register tables** in the word file first **before running the respective programs in visual studio**.
- ❖ The filled memory should represent the memory status of the **executed program**.
- ❖ Memory and register values should be in **hexadecimal format**.
- ❖ Each row in the memory table consists of **16 columns, each of 1 byte**. The row indicates the starting offset. Both the starting offset and the columns are represented with hexadecimal values.
- ❖ Your assignment marks will be calculated from your **submission (60%) + Quiz (30%) + Sessional-I (10%)**
- ❖ You are supposed to debug each question in Visual Studio after filling the tables.
- ❖ You are supposed to submit a **SNAPSHOT** of your visual studio debugging screen, such that it shows the status of registers and memory after debugging. Your submission must include snapshots of each question with a **visible time and date of your computer**. The snapshots for each question must be attached in the word file under respective questions. Remember each snapshot **has marks**.
- ❖ Submit a **PDF** File with the naming format **23i-1234_Section_A1**
- ❖ If you find any **ERROR** in the program highlight the error and write the justification for the error. You must understand ERROR carefully. After identifying the error, you will ignore it by commenting it out and running the program. Also, the register values corresponding to the **erroneous statements** must be left blank. For example:

```
    : mov al, 256 ERROR: 256 is out of range for an 8-bit register like `AL`. It  
    should be within 0-255.
```
- ❖ Data Declarations highlighted in yellow must be replaced as instructed below:

- ✓ 'F1' represents the first character of your first name, 'F2' represents the second character, 'F3' and 'F4' represent the third and the fourth characters of your first name, respectively. If your first name has less characters, then repeat the previous character(s) as needed.
- ✓ 'L1' represents the first character of your last name, 'L2' represents the second character, 'L3' and 'L4' represent the third and the fourth characters of your last name, respectively. If your last name has less characters, then repeat the previous character(s) as needed.
- ✓ 'R' represents the numeric part of your roll number. For example, if your roll number is 23i-1234, then 'R' is 1234. The base for 'R' is represented as (base), where the base can be 'h' for hexadecimal, 'd' for decimal, 'b' for binary, and 'o' for octal. You must first treat 'R' as a decimal number, then convert it into the specified base before substituting the value.

When R is 1234, then $R(d) = 1234d$

$R(h) = 04D2h$

$R(o) = 2322o$

$R(b) = 010011010010b$

- ✓ 'R1' and 'R2' represent the first and the second part of 'R', respectively. For example, if 'R' is 1234, then 'R1' is 12 and 'R2' is 34. The base for 'R1' and 'R2' are represented as (base), where the base can be 'h' for hexadecimal, 'd' for decimal, 'b' for binary, and 'o' for octal. You must first treat 'R1' and 'R2' as decimal numbers, then convert them into the specified bases before substituting the values.
- ✓ 'RR' represents the concatenated roll number. For example, if your roll number is 23i-1234 then 'RR' is 12341234. The "base" rules defined above also apply here.

1. Consider the following Data declaration. Fill in the given Memory for following declaration. **Look for errors if any.**

ASM Program																
<pre> .data valb_1 BYTE 'F1' valb_2 BYTE 1111d valb_3 DB +127 valb_3a DB 256 valb_4 BYTE -128 valb_4a BYTE -129 valb_4b SBYTE -128 valb_5 BYTE 255 valb1_6 SBYTE -129 valb1_7 SBYTE +127 valb_8 SBYTE 'F2' valb1_9 SBYTE -Ah .code main Proc mov edi,offset valb_1 mov al,valb_1 main ENDP END main </pre>																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																

2. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA
<pre> .data DB -5 List1 DB 10, 041h , 'A', 00111111b List2 BYTE 1,2,3,4 BYTE R1(b),R2(b),5 </pre>

```
string5 \
          DB "Hello again"
list3 db 1,2, \
        3,4
```

CODE	REGISTER
.code	
main Proc	ED I
mov edi,Offset List1	AL
mov al,list1-1	AL
mov al,List1+5	BL
mov bl,string+4	CL
mov cl,string2+3	DL
mov dl,string3+5	AH
mov ah,string4+1	BH
mov bh,string5+4	CH
mov ch,string7+3	
main ENDP	
END main	

MEMORY

[illegible]

00404 040																
00404 050																
00404 060																
00404 070																

3. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA																																																																																				
.data list1 BYTE 1, R1(b) list2 BYTE 3 DUP(-2, R2(b)) list3 DB 2 DUP(?, 1, 'F1') list4 BYTE 3 DUP('AB') list5 BYTE 2 DUP(FNAME)																																																																																				
CODE											REGISTER																																																																									
.code main Proc mov edi, offset list1 mov al, List2+2 mov bl, List3+4 mov cl, List4+3 mov dl, List5+3 main ENDP END main											<table border="1" style="width: 100%; border-collapse: collapse;"> <tr><td style="text-align: center; padding: 2px;">EDI</td><td style="width: 50px;"></td></tr> <tr><td style="text-align: center; padding: 2px;">AL</td><td></td></tr> <tr><td style="text-align: center; padding: 2px;">BL</td><td></td></tr> <tr><td style="text-align: center; padding: 2px;">CL</td><td></td></tr> <tr><td style="text-align: center; padding: 2px;">DL</td><td></td></tr> </table>						EDI		AL		BL		CL		DL																																																											
EDI																																																																																				
AL																																																																																				
BL																																																																																				
CL																																																																																				
DL																																																																																				
MEMORY																																																																																				
<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <th style="width: 10%;"></th> <th style="width: 5%;">0</th><th style="width: 5%;">1</th><th style="width: 5%;">2</th><th style="width: 5%;">3</th><th style="width: 5%;">4</th><th style="width: 5%;">5</th><th style="width: 5%;">6</th><th style="width: 5%;">7</th><th style="width: 5%;">8</th><th style="width: 5%;">9</th><th style="width: 5%;">A</th><th style="width: 5%;">B</th><th style="width: 5%;">C</th><th style="width: 5%;">D</th><th style="width: 5%;">E</th><th style="width: 5%;">F</th> </tr> <tr> <td style="text-align: center; padding: 2px;">00404 000</td> <td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td> </tr> <tr> <td style="text-align: center; padding: 2px;">00404 010</td> <td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td> </tr> <tr> <td style="text-align: center; padding: 2px;">00404 020</td> <td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td> </tr> </table>																		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	00404 000																	00404 010																	00404 020																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F																																																																				
00404 000																																																																																				
00404 010																																																																																				
00404 020																																																																																				

4. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA

.data

```
word1 DW 1
word2 DW -1
word3 SWORD -1
word4 SWORD +1
word5 DW 'AB'
word6 DW 'ABC'
list1 DW 2 DUP(R(h))
list2 DW R(o),2
list3 WORD 3 DUP(0)
list4 WORD 2 DUP(0AB12H)
List5 WORD 01F2BH,01101011001011b, R(d)
list6 DW 3 DUP('AB')
```

CODE

REGISTER

.code

```
main Proc
    mov edi,offset list1
    mov al, word1
    mov bx, List1+4
    mov cx, List2+1
    mov dx, List4+2
    mov ax, List6+1
    mov cx, word5+2
    mov cx, word5+4
main ENDP
END main
```

EDI	
AL	
BX	
CX	
DX	
AX	
CX	
CX	

MEMORY

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																
00404010																
00404020																
00404030																

5. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA

.data

```
dword1 DD 1
    dword2 DD -1
    dword3 DWORD -1
    dword4 DWORD 65536
    dword5 DD 'XY'
    dword6 DD 'EFGH'
    List1 DWORD 012345678H,0011010011101011001011b,65
    list4 DWORD 2 DUP(0AB123456H)
```

CODE	REGISTER
.code	
main Proc	EDI
mov edi,Offset dword1	EBX
mov ebx, dword2	ECX
mov ecx, dword5	EDX
mov edx, dword6	EAX
mov eax, List1+4	EBX
mov ebx, list4+2	ECX
mov ecx, List1+8	ECX
mov ecx, dword5+4	
main ENDP	
END main	

MEMORY	
---------------	--

[illegible]

6. Consider the following Data declaration. Fill in the given Memory for following declaration. **Look for errors if any.**

DATA	
------	--

```
.data
v db 0
    tval1 TBYTE 01234H
    tval2 DT 80000000000000001234h
    tval3 TBYTE +1234
    tval4 TBYTE 1
    tval5 TBYTE -1
```

val1 db 0EEH																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																
00404010																
00404020																
00404030																

7. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA															
<pre> .data v3 db count count=2*2+4-2 v1 db count count= R2(b) v2 db count count2= 'A' v3 db count2 count2='ABCD' count3= 'F1 F2 F3 F4 L1' ;5 characters from your name ;no space v4 dd count2 fcount=2.5 CountMax=4294967296 CountMax=4294967294 </pre>															
CODE	REGISTER														
<pre> .code main Proc mov edi,Offset v1 count = 5 mov ax,0 mov al,v1 mov bl,v2 mov cl,count count=257 mov cl,count </pre>	<table> <tr><td>EDI</td><td></td></tr> <tr><td>AX</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>CX</td><td></td></tr> </table>	EDI		AX		AL		BL		CL		CL		CX	
EDI															
AX															
AL															
BL															
CL															
CL															
CX															

<pre> mov cx,count main ENDP END main </pre>	
--	--

MEMORY	
--------	--

[illegible]

8. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA	
------	--

```
.data
firstoffset dd $
list1 db -1,-2,-3, 'F1'
list1size= $-list1
list2 dw 1,2, R2(o),4,5, R(h)
list2size= ($-list2)/2
list3 dd R2(b),2,0Bh
list3size= ($-list3)/4
```

```
.data
firstoffset dd $
list1 db -1,-2,-3, 'F1'
list1size= $-list1
list2 dw 1,2, R2(o),4,5, R(h)
list2size= ($-list2)/2
list3 dd R2(b),2,0Bh
list3size= ($-list3)/4
```

CODE	REGISTER
------	----------

CODE	REGISTER
------	----------

<pre> .code main Proc mov edi,Offset firstoffset mov ah,list1+3 mov bh,list1size mov cx,list2 mov dh,list2size mov dL,list3size mov esi,\$ main ENDP END main </pre>	EDI	
	AH	
	BH	
	CX	
	DH	
	DL	
	ESI	

<pre> .code main Proc mov edi,Offset firstoffset mov ah,list1+3 mov bh,list1size mov cx,list2 mov dh,list2size mov dL,list3size mov esi,\$ main ENDP END main </pre>	EDI	
	AH	
	BH	
	CX	
	DH	
	DL	
	ESI	

<pre> .code main Proc mov edi,Offset firstoffset mov ah,list1+3 mov bh,list1size mov cx,list2 mov dh,list2size mov dL,list3size mov esi,\$ main ENDP END main </pre>	EDI	
	AH	
	BH	
	CX	
	DH	
	DL	
	ESI	

MEMORY

[illegible]

00404																	
020																	

9. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA																																		
.data <pre> v1 db 1 exp equ 2*5 exp=75 exp equ 9 txt equ <"ABC"> txt equ <"abc"> txt2 equ <10*10> symb equ exp mv equ mov axmov equ <mv al,v1> axmove equ <mov al,v1> VAL1 db 6 list db exp,txt,txt2,symb axmov equ <mv bl,v1> </pre>																																		
CODE										REGISTER																								
.code <pre> main Proc mov edi,Offset VAL1 mov ax,0 mov al,v1 mov dx,txt2 axmov axmove axmov equ <mv cl,v1> axmov exp equ 255 mov BX,exp main ENDP END main </pre>										<table border="1" style="width: 100%; border-collapse: collapse;"> <tr><td style="text-align: center;">EDI</td><td></td></tr> <tr><td style="text-align: center;">AX</td><td></td></tr> <tr><td style="text-align: center;">AL</td><td></td></tr> <tr><td style="text-align: center;">DX</td><td></td></tr> <tr><td style="text-align: center;">BL</td><td></td></tr> <tr><td style="text-align: center;">AL</td><td></td></tr> <tr><td style="text-align: center;">CL</td><td></td></tr> <tr><td style="text-align: center;">BX</td><td></td></tr> </table>								EDI		AX		AL		DX		BL		AL		CL		BX		
EDI																																		
AX																																		
AL																																		
DX																																		
BL																																		
AL																																		
CL																																		
BX																																		
MEMORY																																		
<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td style="width: 10%;"></td> <td style="width: 5%;">0</td> <td style="width: 5%;">1</td> <td style="width: 5%;">2</td> <td style="width: 5%;">3</td> <td style="width: 5%;">4</td> <td style="width: 5%;">5</td> <td style="width: 5%;">6</td> <td style="width: 5%;">7</td> <td style="width: 5%;">8</td> <td style="width: 5%;">9</td> <td style="width: 5%;">A</td> <td style="width: 5%;">B</td> <td style="width: 5%;">C</td> <td style="width: 5%;">D</td> <td style="width: 5%;">E</td> <td style="width: 5%;">F</td> </tr> </table>																			0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F																		

00404000																	
00404010																	

10. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA													
.data v1 db 1 exp textequ %(2*5) exp textequ %(20) txt textequ <"Message"> txt2 textequ txt move textequ <MOV> axmov textequ < move al,v1> msg db txt,txt2													
CODE	REGISTER												
.code main Proc mov edi,Offset v1 mov al,v1 axmov mov al,exp exp textequ %(30) axmov textequ < move bl,v1> axmov mov Cl,exp main ENDP END main	<table> <tr><td>EDI</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> </table>	EDI		AL		AL		AL		BL		CL	
EDI													
AL													
AL													
AL													
BL													
CL													

11. Consider the following Data declaration. Fill in the given Memory and register values. **Look for errors if any.**

DATA
.data val1 db 1001b val2 db -10010011y val3 dw 1.0 val4 db -34o val5 dw 5732q val6 db 2.5

val7 db 1+'a'+4/2 val8 dw 5*3/4*2+8 val9 db -255 val4 dd 1.0 wval WORD 0 result word ?																																																																			
CODE											REGISTER																																																								
.code main Proc mov edi,Offset val1 mov ax,3.5 mov bl,val2*2 mov al,2*(128+1) mov cl, val3-1 mov cl,val1+val1a mov al,-45o mov ax,'a' mov ax,'ab' mov ax,AB12h mov al,5*(-3) mov al,-(5*3) mov al,(-3)5 mov al,val3 mov bx,val1 add bx,al mov result,bx mov result,bl add val1,val2 mov 3,val2 main ENDP END main											<table><tr><td>EDI</td><td></td></tr><tr><td>AX</td><td></td></tr><tr><td>BL</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>CL</td><td></td></tr><tr><td>CL</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>AX</td><td></td></tr><tr><td>AX</td><td></td></tr><tr><td>AX</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>AL</td><td></td></tr><tr><td>BX</td><td></td></tr><tr><td>BX</td><td></td></tr></table>						EDI		AX		BL		AL		CL		CL		AL		AX		AX		AX		AL		AL		AL		AL		BX		BX																				
EDI																																																																			
AX																																																																			
BL																																																																			
AL																																																																			
CL																																																																			
CL																																																																			
AL																																																																			
AX																																																																			
AX																																																																			
AX																																																																			
AL																																																																			
AL																																																																			
AL																																																																			
AL																																																																			
BX																																																																			
BX																																																																			
MEMORY																																																																			
<table><tr><td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td></tr><tr><td>00404000</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>00404010</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>																		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	00404000																	00404010																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F																																																			
00404000																																																																			
00404010																																																																			

12. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																

14. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA																							
.data bary BYTE 0ABh,0450,R2(h),25T,50q wary WORD 3456O,R(h),'cd' Dary dd RR(o),'ABcd','L1L2L3L4',0AB12CD34h Qary dq 12345678abcdefh,'AbCd'																							
CODE	REGISTER																						
.code main Proc mov al,bary mov bl,bary+1 mov cl,bary+2 mov dl,bary+3 mov ax,wary mov bx,wary+2 mov cx,wary+6 mov dx,wary+12 mov eax,dary+4 mov ebx,wary+8 mov edx,wary main ENDP END main	<table border="1"> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>DL</td><td></td></tr> <tr><td>AX</td><td></td></tr> <tr><td>BX</td><td></td></tr> <tr><td>CX</td><td></td></tr> <tr><td>DX</td><td></td></tr> <tr><td>EAX</td><td></td></tr> <tr><td>EBX</td><td></td></tr> <tr><td>EDX</td><td></td></tr> </table>	AL		BL		CL		DL		AX		BX		CX		DX		EAX		EBX		EDX	
AL																							
BL																							
CL																							
DL																							
AX																							
BX																							
CX																							
DX																							
EAX																							
EBX																							
EDX																							

15. Consider the following Data declaration. Fill in the given Memory for following declaration. **Look for errors if any.**

DATA
.data bval1 DB 1,2, R2(d) Align 2 wval2 DW R(b) bval2 db R1(d) Align dword

dval3 DD RR(h) Align 4 val4 DB 'F1' ALIGN WORD val5 DB 'L1' pary DD bval1																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																
00404010																
00404020																

16. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA											
.data bval1 DB 034h, R1(b) wval2 DW R(h) dval3 DD RR(b)											
CODE	REGISTER										
.code main Proc mov esi,OFFSET bval1 mov ax, WORD PTR bval1 mov al, BYTE PTR wval2 mov bx, WORD PTR dval3 mov cx, WORD PTR [dval3+2] main ENDP END main	<table border="1"> <tr><td>ESI</td><td></td></tr> <tr><td>AX</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>BX</td><td></td></tr> <tr><td>CX</td><td></td></tr> </table>	ESI		AX		AL		BX		CX	
ESI											
AX											
AL											
BX											
CX											

17. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA	
.data bval1 DB 'F1' wval2 DW 0ABCDH,'F2'	

dval3 DD RR(h) qval4 QWORD 01243567812435678H tval5 tbyte -1											
CODE	REGISTER										
.code main Proc mov al, TYPE bval1 mov bl, TYPE wval2 mov cl, TYPE dval3 mov dl, TYPE qval4 mov ah, TYPE tval5 main ENDP END main	<table border="1"> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>DL</td><td></td></tr> <tr><td>AH</td><td></td></tr> </table>	AL		BL		CL		DL		AH	
AL											
BL											
CL											
DL											
AH											

18. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA															
.data bAry BYTE 010H,020h,030H wAry WORD 5 DUP(?),0 ,0 string1 BYTE 0FFH,0FFH dAry DWORD 2 DUP(3 DUP(0)),0 string2 BYTE 'ABCDEFGHIJKLMNOPQRSTUVWXYZ',0 str2len= \$-string2 bAry2 BYTE 1,2,3, 4,5,6 BYte 4,5,6 r dq 2 dup(5),2 dup('abcd')															
CODE	REGISTER														
.code main Proc mov bl, LENGTHOF bAry mov cl, LENGTHOF wAry mov dl, LENGTHOF dAry mov ah, LENGTHOF string2 mov bh ,str2len mov al, LENGTHOF bAry2 mov bl, LENGTHOF r main ENDP END main	<table border="1"> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>DL</td><td></td></tr> <tr><td>AH</td><td></td></tr> <tr><td>BH</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> </table>	BL		CL		DL		AH		BH		AL		BL	
BL															
CL															
DL															
AH															
BH															
AL															
BL															

19. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA													
.data bAry BYTE 010H,020h,030H wAry WORD 5 DUP(?),0,0 string1 BYTE 0FFH,0FFH dAry DWORD 2 DUP(3 DUP(0)),0 string2 BYTE 'ABCDEFGHIJKLMNOPQRSTUVWXYZ',0 r dq 2 dup(5),2 dup('abcd')													
CODE	REGISTER												
.code main Proc mov bl, SIZEOF bAry mov cl, SIZEOF wAry mov dl, SIZEOF dAry mov ah, SIZEOF string1 mov al, SIZEOF string2 mov bl, SIZEOF r main ENDP END main	<table border="1"> <tr><td>BL</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>DL</td><td></td></tr> <tr><td>AH</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>BL</td><td></td></tr> </table>	BL		CL		DL		AH		AL		BL	
BL													
CL													
DL													
AH													
AL													
BL													

20. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA							
.data valb LABEL Byte valw LABEL WORD vald LABEL DWORD valq LABEL dq valt Tbyte 12345678abcdef123456h							
CODE	REGISTER						
.code main Proc mov al,valb mov bx,valw mov ecx,vald main ENDP END main	<table border="1"> <tr><td>AL</td><td></td></tr> <tr><td>BX</td><td></td></tr> <tr><td>ECX</td><td></td></tr> </table>	AL		BX		ECX	
AL							
BX							
ECX							

- [illegible]

22. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA																																					
.data Ary1 BYTE 'F1',2, R2(h),4 ary2 WORD 'L1',05678h, R(o),567o ary3 DWORD 'L2',012345678h, RR(h),567o																																					
CODE	REGISTER																																				
.code main Proc mov ESI,0 mov al,ary1[ESI] mov ESI,1 * TYPE ary1 mov bl,ary1[ESI] mov ESI,2 * TYPE ary1 mov cl,ary1[ESI] mov ESI,0 mov ax,ary2[ESI] mov ESI,1 * TYPE ary2 mov bx,ary2[ESI] mov ESI,2 * TYPE ary2 mov cx,ary2[ESI] mov ESI,0 mov eax,ary3[ESI] mov ESI,1 * TYPE ary3 mov ebx,ary3[ESI] mov ESI,2 * TYPE ary3 mov ecx,ary3[ESI] main ENDP END main	<table border="1"> <tr><td>ESI</td><td></td></tr> <tr><td>AL</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>BL</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>CL</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>AX</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>BX</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>CX</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>EAX</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>EBX</td><td></td></tr> <tr><td>ESI</td><td></td></tr> <tr><td>ECX</td><td></td></tr> </table>	ESI		AL		ESI		BL		ESI		CL		ESI		AX		ESI		BX		ESI		CX		ESI		EAX		ESI		EBX		ESI		ECX	
ESI																																					
AL																																					
ESI																																					
BL																																					
ESI																																					
CL																																					
ESI																																					
AX																																					
ESI																																					
BX																																					
ESI																																					
CX																																					
ESI																																					
EAX																																					
ESI																																					
EBX																																					
ESI																																					
ECX																																					

23. Consider the following Data declaration. Fill in the given register values. **Look for errors if any.**

DATA	
.data ary WORD 'L1',05678h, R(b),567o	
CODE	REGISTER

.code	
main Proc	
mov ESI,0	ESI
mov ax,ary[ESI * TYPE ary]	AX
INC ESI	ESI
mov BX,ary[ESI * TYPE ary]	BX
INC ESI	ESI
mov CX,ary[ESI * TYPE ary]	CX
INC ESI	ESI
main ENDP	
END main	

24. Write an assembly program that reverses the following string in the memory. Attach screenshot. **Only 'jmp' and labels are allowed:**

'!egaugnal ylbmessA eroda I'

25. Implement following C code in assembly language. Attach screenshot. **You can use jmp/labels and loop:**

```
int si=0;
for(int a=0;a<=3;a++)
{
    for(int b=a;b<=8;b=b+2)
    {
        si++;
    }
}
```

26. Write code that initializes the following array with the first 100 integers using INDIRECT ADDRESSING. You must upload a screenshot of memory. **Only 'jmp' and labels are allowed:**

```
ary dword 100 dup(0abcdef12h)
```

27. Write an assembly program that converts your name from lowercase to uppercase. You must update the name in the memory and attach a screenshot of it. **You can use jmp/labels or loop.**

28. Write an assembly language program that calculates factorial for any number. Attach screenshot as well. **You can use jmp/labels or loop.**